

Theory of Operation

Academic Integrity Submission & Similarity Analysis System

Group 16 — Last Minute Warriors

COSC 4P02

Version 1.0

April 23, 2026

Revision History

Version	Date	Author / Team	Description
1.0	April 23, 2026	Group 16 — Last Minute Warriors	Initial theory-of-operation document generated from the current repository, including web, API, worker, database schema, storage, engine, tests, configuration, and README behavior.

Contents

Revision History	1
1 Introduction	4
2 System Purpose	4
3 Operational Overview	4
4 System Architecture	5
4.1 Frontend / Client Application	5
4.2 Backend / API Server	5
4.3 Database / Metadata Layer	7
4.4 File or Object Storage Layer	7
4.5 Similarity Analysis Engine	7
4.6 Background Worker	7
4.7 Component Communication	7
4.8 Student Submission Upload	7
4.9 File Validation	8
4.10 Metadata Creation	8
4.11 Storage of Uploaded Files	8
4.12 Template Processing	9
4.13 Analysis Job Execution	9
4.14 Similarity Result Generation	9
4.15 Evidence Retrieval	9

5 Engine Theory of Operation	9
5.1 Engine Implementation Language and Invocation	9
5.2 Supported Input Languages	10
5.3 Source File Discovery	10
5.4 Parsing and Tokenization	10
5.5 Normalization	10
5.6 Feature Generation and Fingerprints	11
5.7 Pairwise Comparison Process	11
5.8 Similarity Algorithm	11
5.9 Score Calculation	12
5.10 Evidence-Span Generation	12
5.11 Output Format	12
6 Template Subtraction	13
7 Repository Comparison Model	13
8 Similarity Score Interpretation	13
9 Analysis Job Lifecycle	14
10 Privacy and Anonymization Operation	14
11 Interface Operation	15
11.1 Student Upload	15
11.2 Instructor Setup	16
11.3 Running Analysis	16
11.4 Viewing Results	16
11.5 Pair Comparison View	16
12 Failure Handling and Edge Cases	16
13 Operational Assumptions	17
14 Limitations	17
15 Verification and Testing	18

15.1 Engine Tests	18
15.2 API, Worker, and Frontend Tests	19
15.3 Manual Verification Paths	19
16 Glossary	19
A Appendix: Diagrams, Screenshots, and Samples	20

1 Introduction

This document describes the operational theory of the Academic Integrity Submission & Similarity Analysis System, implemented in the repository as Anti-Vibe Coder. It explains how the system works internally at an operational level: how components cooperate, how data moves through the system, how background jobs are executed, and how the similarity engine produces review evidence.

The document supports the Stage 4 requirement to explain “How the Engine works” and the broader “Theory of Operation.” It is not an installation manual or an end-user workflow guide. It focuses on the internal behavior needed by instructors, evaluators, developers, and maintainers to understand why the system works and where the current implementation boundaries are.

2 System Purpose

The system addresses a practical academic-integrity review problem: programming assignments must be collected, normalized, compared, and presented to instructors in a way that supports evidence-based review. The system accepts source-code submissions, prepares relevant source files, compares submissions against allowed peer and historical repositories, and presents matching regions in an instructor-facing web interface.

The current implementation supports review and investigation. It does not automatically determine misconduct. A high similarity score or shared evidence span indicates that an instructor should inspect the pair in context; it is not, by itself, a disciplinary conclusion.

3 Operational Overview

At a high level, the system operates as a web application plus asynchronous analysis pipeline:

1. **Authentication and RBAC.** Professor users authenticate with email and password. Session cookies identify authenticated requests. API guards enforce professor-only endpoints and assignment ownership. Public student upload endpoints are intentionally unauthenticated but protected by assignment keys and upload status tokens.
2. **Assignment setup.** A professor creates an assignment with a title, language, optional due date, and generated assignment key. Offering management is **Planned / Not available in this version**; assignments are managed directly.
3. **Submission upload.** Students submit a zip archive through the public submission portal using an assignment key. Normal public submissions encrypt student identity in the browser before upload.
4. **Template upload.** Professors may upload starter or reference code as a template archive. The newest template version becomes active.
5. **Template subtraction.** During analysis, the engine masks code regions that match the active template so common starter code does not inflate student-to-student similarity.

6. **Analysis job creation.** A professor manually starts a comparison run when enough prepared inputs exist and no active upload or comparison job is running.
7. **Background processing.** Redis-backed BullMQ queues decouple request handling from upload preparation and engine execution.
8. **Similarity scoring.** The Rust engine parses and normalizes sources, compares allowed pairs, calculates code similarity, detects meaningful comment matches, and emits evidence spans.
9. **Result storage.** Pair results and pair matches are persisted in PostgreSQL through Prisma models.
10. **Instructor review.** The web UI displays suspicious pair groups, similarity scores, comment match counts, and a side-by-side evidence viewer.

4 System Architecture

The repository contains the following implemented components.

4.1 Frontend / Client Application

The frontend is a Next.js application under `apps/web`. It uses React components, TanStack Query for API state, and Monaco Editor for the side-by-side evidence viewer.

Implemented routes include:

- `/`: public student submission portal.
- `/help/student`: student help guide.
- `/login`: instructor sign-in.
- `/signup`: professor account creation.
- `/professor`: instructor dashboard.
- `/professor/help`: instructor help guide.
- `/professor/settings`: professor account settings.
- `/professor/assignments/[assignmentId]`: assignment workspace.
- `/professor/assignments/[assignmentId]/pairs/[pairId]`: pair comparison view.

4.2 Backend / API Server

The backend is a NestJS application using Fastify under `apps/api`. It exposes controllers for authentication, assignments, uploads/submissions, comparisons, and health checks. API requests are validated by controller/service logic and persisted through Prisma.

The API performs synchronous validation and metadata creation, then enqueues asynchronous work for long-running upload preparation or comparison runs.

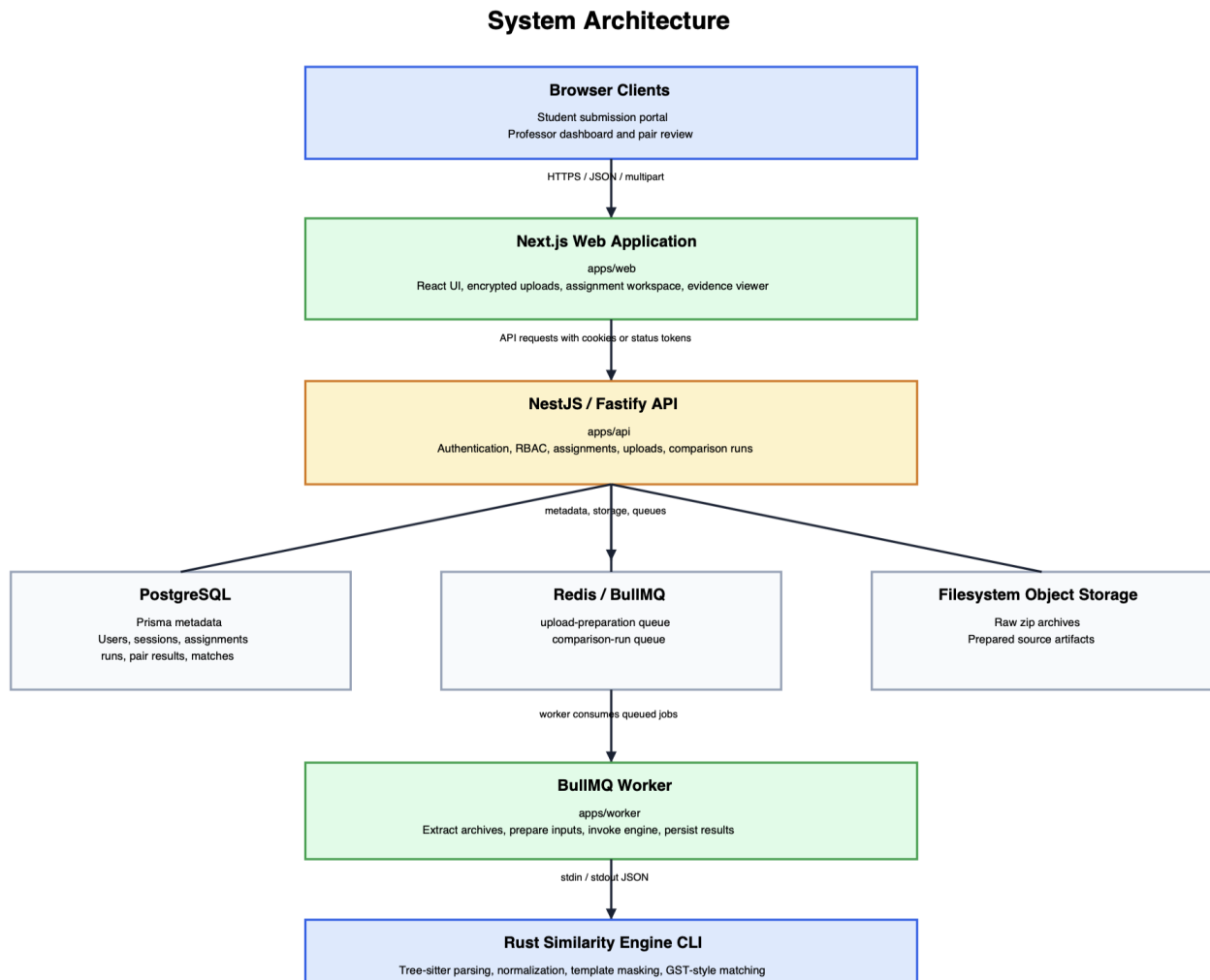


Figure 1: System architecture diagram.

4.3 Database / Metadata Layer

The database layer uses Prisma with PostgreSQL. The schema defines users, sessions, assignments, assignment keys, upload batches, submissions, submission files, template versions, template files, comparison runs, pair results, and pair matches.

The metadata layer records assignment ownership, upload and comparison status, object-storage keys, prepared source maps, similarity scores, and evidence spans.

4.4 File or Object Storage Layer

The current storage implementation is filesystem-backed object storage, configured by `OBJECT_STORAGE_ROOT`. Uploaded archives are staged, promoted into raw upload storage, and prepared source files are stored under generated object keys. The Docker Compose deployment shares the object-storage volume between the API and worker services.

Object storage modes beyond `filesystem` are **Planned / Not available in this version**.

4.5 Similarity Analysis Engine

The similarity engine is implemented in Rust under `engine/`. It is split into crates for contracts, language parsing, Greedy String Tiling-style matching, core analysis, and a command-line interface. The engine CLI reads an analysis request as JSON from standard input and writes an analysis response as JSON to standard output.

4.6 Background Worker

The worker service under `apps/worker` consumes BullMQ queues using Redis. It runs two job types:

- **upload-preparation**: extracts and prepares uploaded archives.
- **comparison-run**: invokes the Rust engine and persists comparison results.

Worker concurrency is configurable through `WORKER_UPLOAD_CONCURRENCY` and `WORKER_COMPARISON_CONCURRENCY`.

4.7 Component Communication

The browser communicates with the API over HTTP. The API stores metadata in PostgreSQL, stores uploaded archives through the object-storage abstraction, and enqueues jobs in Redis. The worker reads jobs from Redis, reads and writes object-storage artifacts, updates PostgreSQL status records, and invokes the local Rust engine binary through a child process.

4.8 Student Submission Upload

For normal public student submissions, the web client fetches the public submission identity key, encrypts the student's identity payload in the browser using RSA-OAEP with SHA-256, then uploads a multipart form containing the assignment key, encrypted identity string, and zip file.

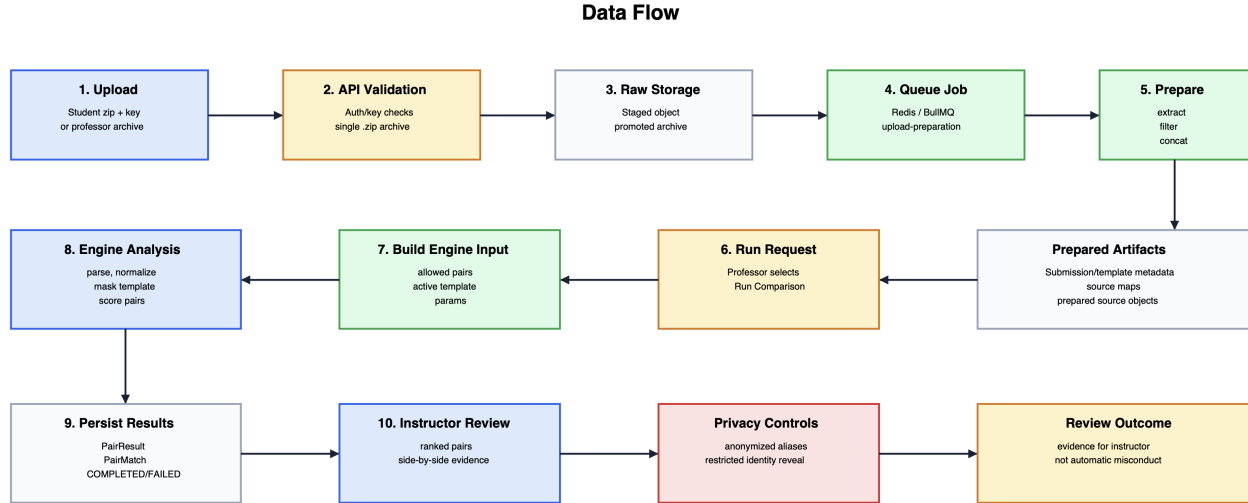


Figure 2: Data-flow diagram

The API validates the zip filename, normalizes the assignment key, finds an active assignment key, stages the uploaded file, creates an upload batch with purpose `STUDENT_SUBMISSION`, and enqueues upload preparation. It returns an upload batch identifier and public status token.

4.9 File Validation

Initial API validation checks that an archive file is present, only one archive file was supplied, required multipart fields exist, and the filename ends with `.zip`. Worker validation extracts the archive and rejects invalid zip data, unsafe archive paths, excessive nesting, excessive expanded size, excessive source-file count, unsupported bulk/historical archive shapes, and archives with no relevant source files.

4.10 Metadata Creation

Upload metadata is recorded as an `UploadBatch`. Once prepared, current or historical submissions are recorded as `Submission` rows with source maps and file records. Template uploads create `AssignmentTemplate` and `TemplateFile` rows. Comparison runs create `ComparisonRun`, `PairResult`, and `PairMatch` rows.

4.11 Storage of Uploaded Files

The original zip archive is stored as a raw object. Prepared source files are stored separately after extraction. Download operations reconstruct zip archives from prepared source artifacts, not necessarily from every file in the original upload.

4.12 Template Processing

Template uploads follow the same extraction and source filtering path as submissions, but they are persisted as template versions rather than suspicious submissions. The newest template version is marked active.

4.13 Analysis Job Execution

When a professor starts a run, the API checks assignment ownership, active upload count, active comparison count, and input sufficiency. A run requires at least one current submission and at least two non-template submissions total. The API creates a queued comparison run and enqueues the job.

4.14 Similarity Result Generation

The worker builds an engine request containing the assignment language, prepared submissions, optional active template, allowed pairs, and engine parameters. The Rust CLI receives that JSON, analyzes each pair, and returns pair results. The worker persists the response in the database and marks the run completed.

4.15 Evidence Retrieval

The instructor UI fetches assignment details, pair summaries, and pair details from the API. Pair detail responses include concatenated source, source maps, file metadata, match spans, similarity score, matched token count, and comment match count.

5 Engine Theory of Operation

The engine is the core comparison component. It is implemented in Rust and invoked locally by the worker. It does not use artificial intelligence, external model calls, or outsourced comparison services.

5.1 Engine Implementation Language and Invocation

The engine workspace is under `engine/`. The command-line entry point is `engine/crates/engine-cli/src/main.rs`. The worker spawns the executable specified by `ENGINE_BINARY_PATH`, writes a JSON `EngineRunRequest` to standard input, reads JSON from standard output, and treats a non-zero process exit as a comparison failure.

The CLI request and response use schema version 1.0. The response includes the engine version, language, pair results, similarity scores, matched token counts, and evidence matches.

5.2 Supported Input Languages

The current engine supports:

- Java, parsed with `tree-sitter-java`.
- C, parsed with `tree-sitter-c`.
- C++, parsed with `tree-sitter-cpp`.

The web and API assignment language values are `java`, `c`, and `cpp`. Other languages are not implemented.

5.3 Source File Discovery

Source discovery occurs in the worker before engine invocation. Archives are extracted, ignored system junk is skipped, and files are filtered by assignment language:

- Java: `.java`.
- C: `.c`, `.h`.
- C++: `.cpp`, `.cc`, `.cxx`, `.hpp`, `.hh`, `.hxx`, `.h`.

Relevant files are sorted deterministically by relative path and concatenated. A source map records each file's byte range in the concatenated source so later evidence spans can be mapped back to file paths.

5.4 Parsing and Tokenization

The engine parses each concatenated source string with `tree-sitter`. It walks the parse tree and separates code tokens from comment tokens. Whitespace is ignored.

The tokenization process is syntax-aware but does not perform semantic analysis, data-flow analysis, type checking, or compilation.

5.5 Normalization

The engine normalizes source before comparison to reduce sensitivity to superficial edits.

Language / Item	Current Normalization Behavior
Java identifiers	<code>identifier</code> becomes <code>ID</code> ; <code>type_identifier</code> becomes <code>TYPE_ID</code> .
Java literals	String literals become <code>STR</code> ; character literals become <code>CHAR</code> ; numeric literals become <code>NUM</code> ; boolean values become <code>BOOL</code> .
Java boilerplate	<code>package_declaration</code> and <code>import_declaration</code> subtrees are skipped and treated as structural gaps.

C / C++ identifiers	<code>identifier</code> becomes <code>ID</code> ; <code>type_identifier</code> becomes <code>TYPE_ID</code> ; <code>field_identifier</code> becomes <code>FIELD_ID</code> ; C++ <code>namespace_identifier</code> becomes <code>NS_ID</code> .
C / C++ literals	String-like literals become <code>STR_LIT</code> ; character literals become <code>CHAR_LIT</code> ; number literals become <code>INT_LIT</code> or <code>FLOAT_LIT</code> .
C / C++ preprocessor	Node kinds beginning with <code>preproc_</code> are skipped and treated as structural gaps.
Comments	Comments are stored separately for comment-match evidence and do not contribute to code-token counts or code similarity thresholds.
Formatting	Whitespace and formatting are ignored by token collection.

5.6 Feature Generation and Fingerprints

The current implementation does not generate persistent hash fingerprints and does not implement winnowing. Its primary comparison feature is the normalized token sequence produced from syntax parsing. The GST module then identifies non-overlapping common token tiles. Comment matching uses normalized comment text rather than token-tile matching.

5.7 Pairwise Comparison Process

The worker generates allowed pairs before invoking the engine:

- Every current submission is compared against every other current submission.
- Every current submission is compared against every historical submission.
- Historical-vs-historical pairs are not generated.
- Templates are not generated as suspicious submissions.

For each allowed pair, the engine retrieves the parsed left and right submissions, builds template masks if an active template is present, runs token matching, computes score metrics, and emits code and comment evidence.

5.8 Similarity Algorithm

The `engine-gst` crate implements a Greedy String Tiling-style token matcher. It repeatedly finds the longest unmarked common token substrings at least as long as the configured minimum match length, accepts non-overlapping tiles, marks their token ranges, and repeats until no qualifying tile remains.

After initial tile discovery, the core engine applies additional filtering:

- Tiles are split across hidden structural gaps such as skipped imports, preprocessor regions, or file boundaries.

- A meaningful tile threshold is computed as 3% of the smaller active token count, clamped from 10 to 15 tokens.
- Exact tiles meeting that threshold are retained.
- Smaller anchors of at least 6 tokens may be clustered into one evidence region only when the exact matched token total reaches the threshold and gap constraints are satisfied.

5.9 Score Calculation

The final code similarity score is a floating-point value from 0.0 to 1.0. It is based on meaningful code regions after template masking. The engine calculates coverage on both sides, combines harmonic and geometric coverage into a coverage anchor, and multiplies by contiguity, fragmentation, and purity factors. The score is clamped to the range $[0, 1]$.

This scoring design rewards broad, meaningful, contiguous overlap and suppresses tiny scattered matches. Comment matches do not affect the code similarity score.

5.10 Evidence-Span Generation

Code evidence spans are generated from meaningful code regions. Each span records:

- Byte start and byte end.
- Optional file path from the source map.
- Line and column metadata.
- Token start and token end metadata.

Comment evidence spans are generated from matching normalized comments that meet the meaningful length threshold. The engine normalizes comment text by lowercasing, removing punctuation into spaces, collapsing whitespace, and trimming. Template comments are counted and removed from comment matches so starter comments do not become evidence.

5.11 Output Format

The engine output contains:

- `schemaVersion`
- `engineVersion`
- `language`
- `pairResults`
- For each pair: `pairId`, submission IDs, `similarityScore`, `matchedTokenCount`, and `matches`.
- For each match: match ID, kind `code` or `comment`, left span, right span, and matched token count.

6 Template Subtraction

Template subtraction prevents shared starter code from inflating similarity scores. In the current implementation, subtraction is implemented as token masking rather than physical removal from stored files.

When an active template is present, each submission is compared against the parsed template using the same GST-style token matcher with a fixed template mask threshold of 10 tokens. Matching submission tokens are marked as masked. Masked tokens are excluded from active token counts, code-region selection, and score calculation.

Comment subtraction is handled separately. The engine normalizes meaningful template comments and removes matching occurrences from each submission's prepared comment list before comparing comments between submissions.

This behavior is consistent with the goal of discounting shared provided material, but it is important that instructors upload accurate starter code. Incorrect or missing templates may produce higher similarity scores for shared starter regions.

7 Repository Comparison Model

The current repository comparison model is implemented through submission kinds:

- **current**: student submissions for the active assignment.
- **historical**: uploaded prior submissions used as a comparison repository.

The implemented pair-generation model compares current-vs-current and current-vs-historical. It does not compare historical-vs-historical.

The SRS describes offerings and selectable secondary repositories. Those concepts are **Planned / Not available in this version** in the current repository. There is no offering model, no offering management UI, and no repository-selection screen. The implemented substitute is manual historical archive upload through the instructor assignment workspace.

8 Similarity Score Interpretation

A similarity score is an evidence signal that estimates the degree of meaningful normalized code overlap between two submissions after template masking. A high score suggests that the pair deserves instructor review. It does not prove academic misconduct.

Several conditions can produce similarity:

- Common starter code that was not uploaded as a template.
- Required assignment structure.
- Small assignments with limited possible solutions.

- Legitimate collaboration allowed by course policy.
- Copied or jointly developed code.

Instructor review remains required because the engine cannot determine intent, policy context, authorization, or institutional process outcomes.

9 Analysis Job Lifecycle

Comparison runs use the Prisma enum `ComparisonRunStatus`. The actual persisted states are:

State	Meaning	Transition Behavior
QUEUED	The API created the comparison run and enqueued it for the worker.	Created after professor selects Run Comparison .
RUNNING	The worker has started processing the run.	Set before invoking the engine.
COMPLETED	The engine returned successfully and pair results were persisted.	Pair results and pair matches are available to the instructor UI.
FAILED	The worker or engine failed to complete the run.	A user-facing error message is stored, and the assignment may be rerun.

Upload preparation has a parallel lifecycle using `RECEIVED`, `PROCESSING`, `READY`, and `FAILED`. Assignment-level UI statuses such as **not ready**, **preparing**, **ready to run**, **running**, **current**, **stale**, and **failed** are derived from upload counts, comparison-run state, available inputs, and comparison input version.

10 Privacy and Anonymization Operation

Normal public student uploads encrypt identity data in the browser before it reaches the API. The frontend imports the institution public key and uses RSA-OAEP with SHA-256. The encrypted identity payload includes normalized student name, student number, optional email, and assignment key.

Prepared normal public submissions receive alias-style display names such as `SUB-...`. Professors can reveal identity when the encrypted identity is present and decryptable by the current server key. Bulk public uploads do not include encrypted identity; their visible names come from sanitized child zip filenames.

RBAC behavior includes:

- Professor-only endpoints require a valid session cookie.
- Role guards reject users without the required role.
- Assignment services verify professor ownership before returning or mutating assignment data.

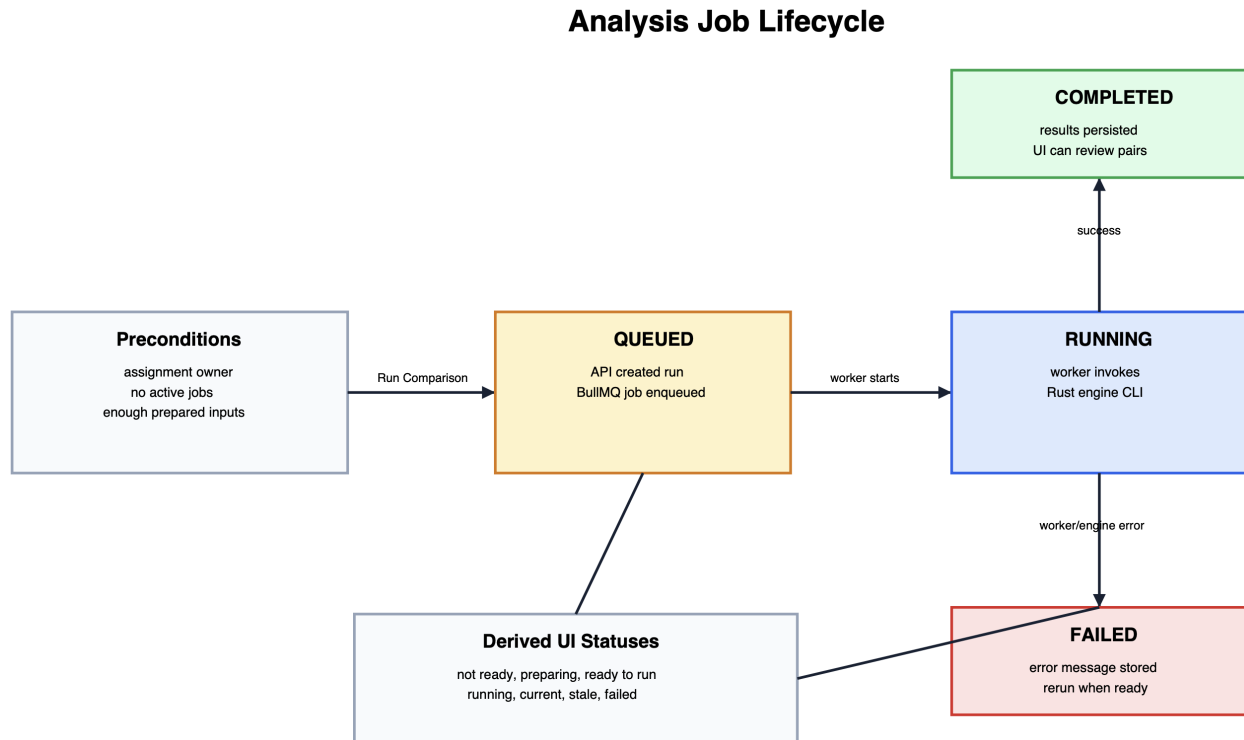


Figure 3: Analysis job lifecycle diagram.

- Public upload status requires a status token derived from the upload batch identifier.

Students can submit and observe the status of their current public upload while the browser has the returned status token. The current UI does not expose student login, persistent student history, or pair results to students.

The repository does not implement a formal audit-log feature. It does store timestamps such as creation/update times, session last-used time, upload statuses, and comparison-run timestamps, but those fields are not presented as an audit dashboard.

11 Interface Operation

The frontend calls the API through `apps/web/lib/api.ts`. API calls use browser credentials for professor sessions and multipart requests for archive uploads.

11.1 Student Upload

The `StudentSubmissionPanel` component collects identity fields, assignment key, and archive file. It encrypts identity using `encryptSubmissionIdentity`, posts to `/uploads/public/student/archive`, and polls `/uploads/public/:uploadBatchId` with the returned status token.

11.2 Instructor Setup

The `ProfessorDashboard` component lists assignments from `/assignments`, creates assignments through `POST /assignments`, displays active assignment keys, and opens the assignment workspace.

11.3 Running Analysis

The `AssignmentWorkspace` component calls `POST /comparison-runs` through `createComparisonRun`. It disables the run action when the assignment is not runnable or a run is already pending.

11.4 Viewing Results

The assignment workspace fetches `/assignments/:assignmentId`. Completed comparison runs include pair summaries that are grouped into **Suspicious by Code**, **Comment-Supported**, and optionally **Remaining Pairs**.

11.5 Pair Comparison View

The `PairViewerPanel` fetches `/comparison-runs/pair/:pairResultId`. It displays metrics, identity panels, and the `EvidenceViewer`. The evidence viewer renders read-only Monaco editors and applies colored decorations to matched spans.

12 Failure Handling and Edge Cases

Condition	Current Behavior	Operational Result
Invalid archive file	API or worker rejects the upload if no file is present, multiple files are present, the file is not a zip, or extraction fails.	Upload batch is rejected or marked FAILED .
Unsupported language	Assignment language is constrained to Java, C, or C++. Source files not matching the assignment language are ignored.	If no supported files remain, preparation fails.
Empty or irrelevant submission	Worker reports no relevant source files.	Upload batch is marked FAILED ; in bulk current uploads, child archives with no supported files may be skipped if at least one valid child remains.
Missing template	Template is optional.	Analysis runs without template masking. Shared starter code may therefore affect scores.

Failed analysis job	Worker catches the failure, stores a generic user-facing comparison failure message, and marks the run FAILED .	Instructor can retry when ready.
Storage or queue failure	Staged/final objects are deleted best-effort. If queueing fails after upload-batch creation, the upload batch is marked failed.	User sees an upload failure or failed batch status.
Authorization failure	Session and role guards reject missing, expired, invalid, or unauthorized requests. Assignment services also check ownership.	API returns authentication or permission errors.
Unsafe archive paths	Archive entries containing unsafe path segments are rejected.	Upload preparation fails.
Active jobs during deletion	Assignment and account destructive actions check for active upload or comparison jobs.	Destructive action is blocked until jobs finish.

13 Operational Assumptions

The current implementation assumes:

- Users have modern browsers with Web Crypto support for encrypted public submissions.
- Instructors create assignments with the correct language and distribute the correct assignment key.
- Uploaded archives contain supported source files and use the expected zip structure.
- The API, worker, PostgreSQL, Redis, filesystem object storage, and Rust engine binary are all available.
- The server has enough CPU, memory, and storage for archive extraction and pairwise analysis.
- The engine runs locally as a deterministic executable and is not outsourced to AI services.
- Instructors provide template code when shared starter material should be discounted.

14 Limitations

Implementation limitations discovered in the repository include:

- Offering management is not implemented.
- Managed repository selection is not implemented; historical uploads are the available repository-like mechanism.

- Formal report export is not implemented.
- Student login and persistent student submission history are not exposed in the web UI.
- Browser-based system administration and audit dashboards are not implemented.
- Only Java, C, and C++ are supported.
- The upload UI accepts zip archives only.
- The engine performs syntax-aware token analysis, not compilation, type checking, semantic equivalence detection, or intent analysis.
- The comparison model is pairwise and local to generated allowed pairs; historical-vs-historical comparison is not performed.
- Filesystem object storage is the only supported storage mode in the current configuration.
- Scalability depends on Redis, PostgreSQL, worker concurrency, local engine runtime, and storage capacity; no distributed engine cluster is implemented.
- No explicit formal audit log is implemented beyond ordinary metadata timestamps and session records.

15 Verification and Testing

The repository validates the theory of operation through unit tests and source-level contracts.

15.1 Engine Tests

Rust tests in `engine/crates/engine-core/src/lib.rs` and `engine/crates/engine-language/src/lib.rs` validate:

- Java, C, and C++ parsing and normalization.
- Identifier and literal normalization.
- Exclusion of Java package/import boilerplate and C/C++ preprocessor boilerplate.
- Template masking and template-heavy assignments.
- Meaningful tile thresholding.
- Cluster behavior for nearby short anchors.
- Comment normalization and template comment exclusion.
- Evidence span byte ranges and source-map file-path lookup.

Engine tests can be run with:

```
cargo test --manifest-path engine/Cargo.toml --workspace
```

15.2 API, Worker, and Frontend Tests

TypeScript tests validate selected API and worker behavior, including:

- Assignment due-date parsing and maintenance behavior.
- Comparison-run preconditions and status derivation.
- Authentication and password behavior.
- Archive extraction and bulk child-zip layout rules.
- Upload preparation artifact behavior.
- Evidence viewer highlighting utilities.
- Filesystem storage staging and promotion.

The root package provides:

```
npm run test --workspaces --if-present
```

15.3 Manual Verification Paths

The README identifies deployment verification paths: validate health endpoints, login, assignment creation, upload preparation, comparison run, and pair viewer flow. These paths exercise the full operational chain from HTTP request through worker processing and result review.

16 Glossary

Term	Definition
Submission	A prepared source-code artifact associated with an assignment. Current submissions are active student submissions; historical submissions are prior submissions used for comparison.
Offering	A course section or delivery instance described by the SRS. Offering management is Planned / Not available in this version in the current implementation.
Assignment	An instructor-created comparison unit with title, language, optional due date, generated key, uploads, templates, and comparison runs.
Template	Starter or reference code uploaded by an instructor. The active template is used for masking during analysis.

Template Subtraction	The masking operation that excludes code and comments matching active template material from similarity evidence and scoring.
Repository	A comparison collection. In this implementation, historical submissions uploaded as a parent zip serve as the repository-like input. Managed repository selection is Planned / Not available in this version.
Analysis Job	A queued comparison run processed asynchronously by the worker and Rust engine.
Tokenization	The process of parsing source and extracting normalized code tokens plus separate comment tokens.
Normalization	The conversion of raw syntax into comparable forms, such as replacing identifiers with ID and literals with placeholder tokens.
Fingerprint	A derived feature used for comparison. The current implementation does not use persistent hash fingerprints or winnowing; normalized token streams and GST tiles are the comparison features.
Similarity Score	A value from 0.0 to 1.0 representing meaningful normalized code overlap after template masking.
Evidence Span	A byte, line, column, and optional file-path region showing where a code or comment match occurs in both submissions.
RBAC	Role-based access control. The API uses session and role guards plus assignment ownership checks.
Anonymized Identifier	A display name such as SUB-... used instead of raw student identity for encrypted public submissions.

A Appendix: Diagrams, Screenshots, and Samples

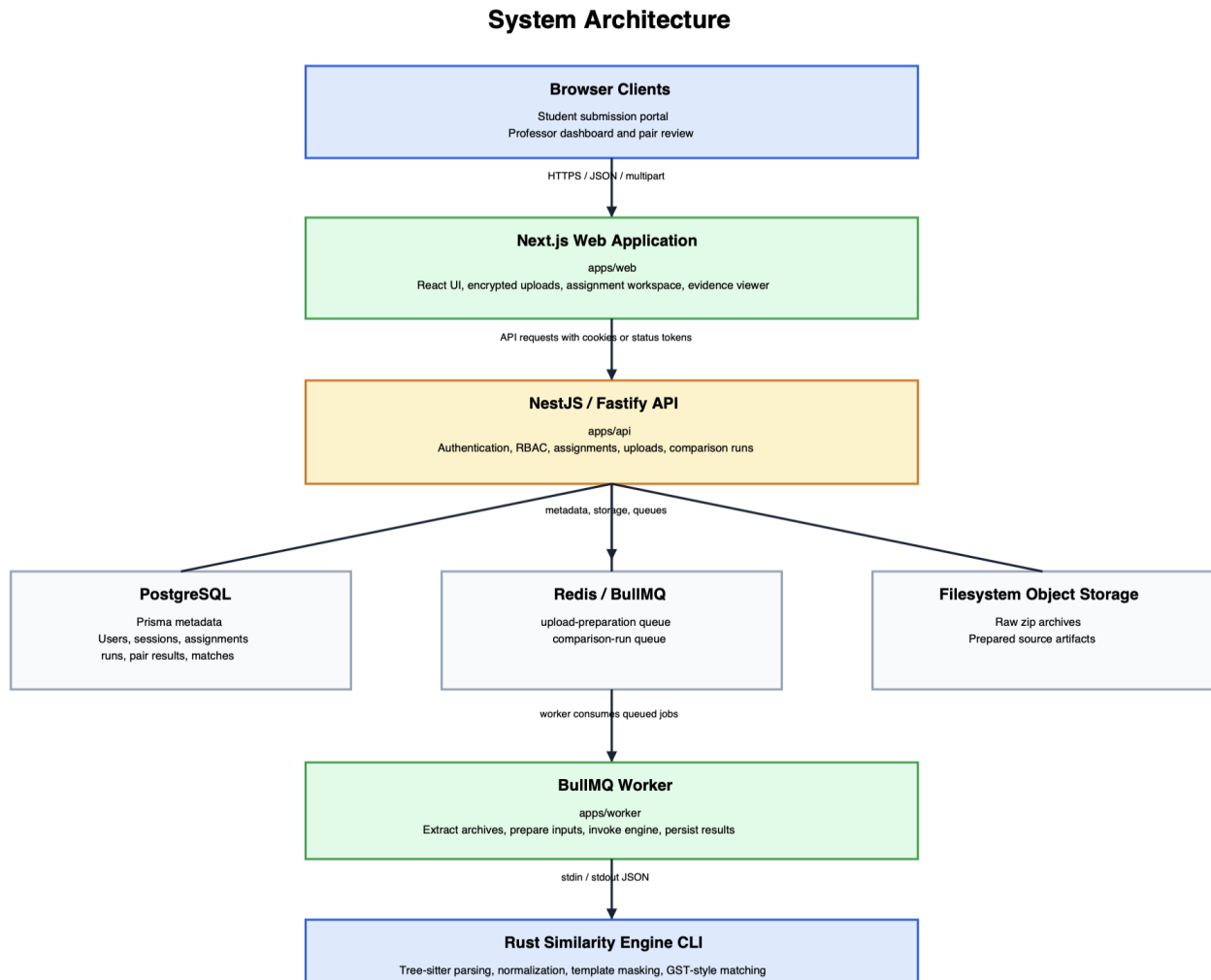


Figure 4: Finalized architecture diagram.

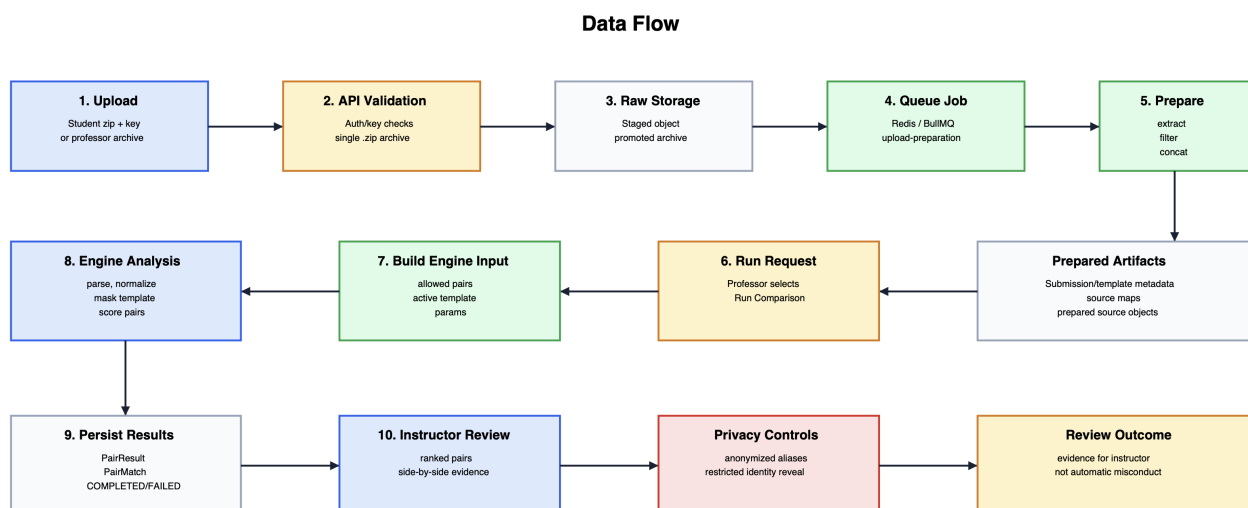


Figure 5: Finalized data-flow diagram.

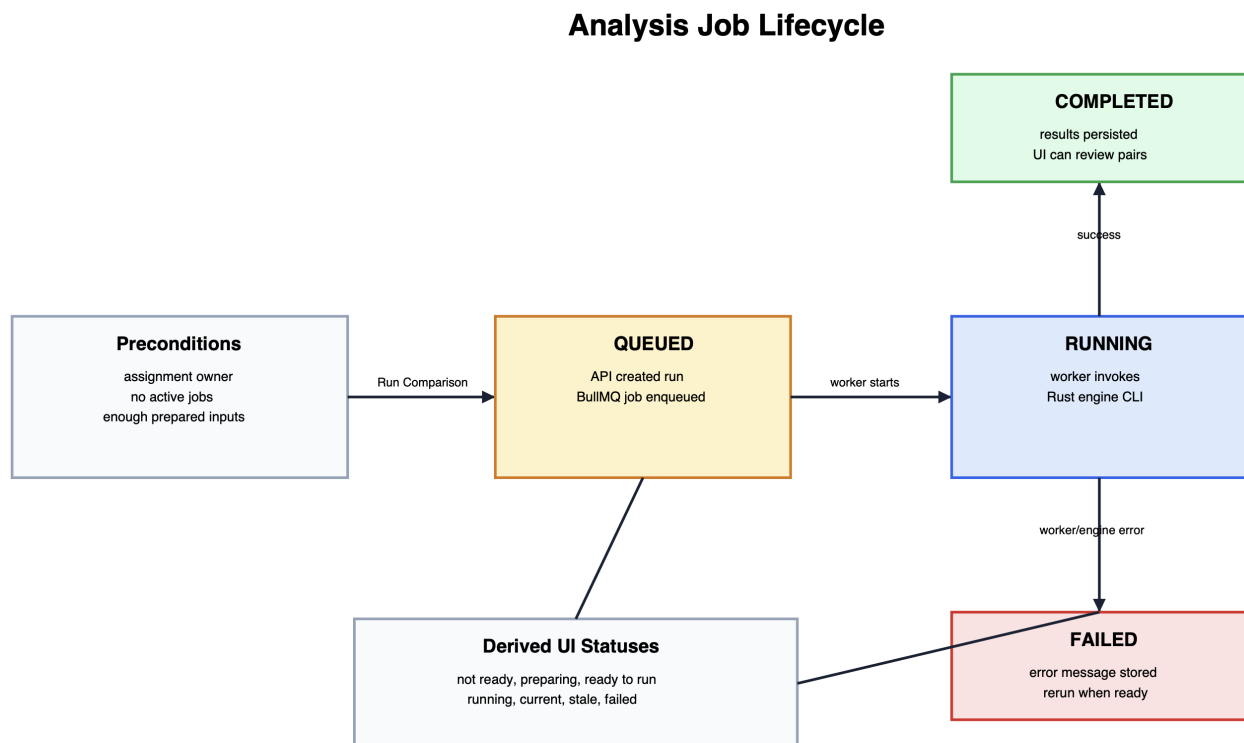


Figure 6: Finalized analysis job life-cycle diagram.

Suspicious by Code
Code similarity $\geq 35.0\%$ 1

LEFT	RIGHT	CODE SIM	COMMENTS	MATCHES	
SUB-B73AD6629A Current	SUB-5780751C7C Current	100.0%	2	3	Open

Figure 7: Suspicious pair table screenshot placeholder.

SUB-B73AD6629A
current - click a highlighted region to jump to its pair

```

2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <stdint.h>
6
7 typedef struct DemoRecord {
8     int id;
9     int category;
10    long value;
11    char label[32];
12 } DemoRecord;
13
14 static void alex_initialize_records(DemoRecord *records, int count) {
15     for (int i = 0; i < count; ++i) {
16         records[i].id = i + 1;
17         records[i].category = (i * 3 + 7) % 17;
18         records[i].value = (long)(i + 1) * 41;
19         snprintf(records[i].label, sizeof(records[i].label), "Alex-%03d", i + 1);
20     }
21 }
22
23 static long alex_prefix_0(DemoRecord *records, int count) {
24     long score = 3;
25     for (int i = 0; i < count; ++i) {
26         if ((records[i].category + 0) % 3 == 0) {
27             score += records[i].value / 2;
28         } else {
29             score -= (records[i].id % 4);
30         }
31         if (strstr(records[i].label, "Alex") != NULL) {
32             score += 1;
33         }
34     }

```

SUB-5780751C7C
current - colors stay matched on both sides

```

2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <stdint.h>
6
7 typedef struct DemoRecord {
8     int id;
9     int category;
10    long value;
11    char label[32];
12 } DemoRecord;
13
14 static void blake_initialize_records(DemoRecord *records, int count) {
15     for (int i = 0; i < count; ++i) {
16         records[i].id = i + 1;
17         records[i].category = (i * 5 + 11) % 17;
18         records[i].value = (long)(i + 1) * 53;
19         snprintf(records[i].label, sizeof(records[i].label), "Blake-%03d", i + 1);
20     }
21 }
22
23 static long blake_prefix_0(DemoRecord *records, int count) {
24     long score = 3;
25     for (int i = 0; i < count; ++i) {
26         if ((records[i].category + 0) % 3 == 0) {
27             score += records[i].value / 2;
28         } else {
29             score -= (records[i].id % 4);
30         }
31         if (strstr(records[i].label, "Blake") != NULL) {
32             score += 1;
33         }
34     }

```

Matches
Select a match to scroll both panes and highlight the paired regions.

Code matches 1 match

< Back Next >

Code 1

Comment matches 2 matches

< Back Next >

● Comment 1 ● Comment 2

Figure 8: Pair comparison evidence viewer screenshot placeholder.

Sample Engine Output

The following JSON follows the current engine response contract and mirrors the Java pair test fixture used by the engine test suite.

```
{
  "schemaVersion": "1.0",
  "engineVersion": "documentation-sample",
  "language": "java",
  "pairResults": [
    {
      "pairId": "left__right",
      "leftSubmissionId": "left",
      "rightSubmissionId": "right",
      "similarityScore": 0.72,
      "matchedTokenCount": 18,
      "matches": [
        {
          "matchId": "code-0",
          "kind": "code",
          "left": {
            "byteStart": 0,
            "byteEnd": 51,
            "filePath": "A.java",
            "position": {
              "lineStart": 1,
              "columnStart": 1,
              "lineEnd": 1,
              "columnEnd": 52
            },
            "tokens": {
              "tokenStart": 0,
              "tokenEnd": 18
            }
          },
          "right": {
            "byteStart": 0,
            "byteEnd": 51,
            "filePath": "B.java",
            "position": {
              "lineStart": 1,
              "columnStart": 1,
              "lineEnd": 1,
              "columnEnd": 52
            },
            "tokens": {
              "tokenStart": 0,
              "tokenEnd": 18
            }
          }
        }
      ]
    }
  ]
}
```

```
        }  
      },  
      "matchedTokenCount": 18  
    }  
  ]  
}  
]
```